VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project Report

Student Name	Hiten Gogia
Registration Number	24BCE10832
Course	Open Source Software | Unit Coverage: 1–5
Chosen Software	LibreOffice
Distribution	Fedora 43
LibreOffice Version	26.2.0.3
GitHub Repository	https://github.com/hitengogia/VITYARTHI_PROJECT_OPEN_SOURCE_24BCE10832
Date of Submission	March 2026

1. Introduction

Most of the software that powers the modern world — the servers that run websites, the tools that developers use every day, the operating systems inside supercomputers and smartphones — was not built by a single company and sold for profit. It was built openly, shared freely, and improved collectively. That is the essence of open source.

This report conducts a structured audit of LibreOffice — a free and open-source office suite developed and maintained by The Document Foundation. The audit goes beyond describing what LibreOffice does: it examines why LibreOffice was created, who built it, what freedoms its license grants to every user, what philosophical values it embodies, and how it has shaped the landscape of office productivity software worldwide.

Alongside the written report, five Bash shell scripts have been developed and tested on Fedora 43. These scripts are not intended to be impressive programs — they are designed to demonstrate a working understanding of how open-source tools operate at the command line, and how automation connects to the philosophy of transparency and sharing that underpins the FOSS movement.

Every line of code any developer will write in their career sits on top of open-source foundations. Understanding the philosophy behind that foundation is not optional — it is part of being a computing professional.

Part A — Origin and Philosophy (Units 1 & 2)

A1. The Problem LibreOffice Was Created to Solve

For most of the 1990s and 2000s, the office productivity space was dominated by a single proprietary force: Microsoft Office. It was powerful, widely adopted, and extraordinarily expensive — particularly for individual users, students, small businesses, and public institutions in developing economies. Educators could not freely distribute it. Governments were locked into vendor-specific file formats. Users who received a .docx file had no way to open it without paying for a Microsoft licence.

OpenOffice.org emerged in 2000 as a free, open-source alternative, born largely from the source code of StarOffice, which Sun Microsystems acquired and released under the LGPL. For nearly a decade, OpenOffice filled a crucial gap — it offered a functional word processor, spreadsheet, presentation tool, and database to anyone who wanted them, at no cost. However, progress was slow, community participation was limited by Sun's centralised governance model, and many contributors felt that their voices and contributions were not being taken seriously.

In 2010, Oracle acquired Sun Microsystems. The open-source community responded with immediate concern. Oracle had a commercial reputation that did not naturally align with volunteer-driven community development. Contributors feared that OpenOffice.org would be neglected, commercialised, or simply allowed to stagnate. Those fears proved well-founded: Oracle showed little interest in fostering the open community that OpenOffice depended on.

In September 2010, a group of leading OpenOffice contributors — including developers, localisation volunteers, and long-time advocates — took a decisive step. They announced the formation of The Document Foundation and forked the OpenOffice codebase to create LibreOffice. The fork was more than a technical decision. It was a philosophical declaration: that office productivity software should belong to the community, not to a corporation. That the people who build a project should govern it. That no single commercial entity should be able to hold a community's work hostage.

LibreOffice was designed to solve three deeply interconnected problems. First, the access problem: millions of people worldwide could not afford commercial office software. LibreOffice gave them a fully functional, professionally capable alternative with zero acquisition cost. Second, the freedom problem: users of proprietary software cannot examine how it works, cannot modify it to fit their needs, and cannot share modifications with others. LibreOffice's licence explicitly grants all of these freedoms. Third, the governance problem: OpenOffice.org had shown how quickly a community project could stall when a single corporation controlled it. The Document Foundation's structure — a non-profit governed by elected committees — was designed from the start to prevent any single company or individual from seizing control.

Today, LibreOffice is used by hundreds of millions of people across the world, from home users to governments, schools, and enterprises. Several national and regional governments — including Germany's federal IT agency and municipalities across France and Italy — have migrated from Microsoft Office to LibreOffice as a matter of digital sovereignty. The software is translated into over 100 languages. Every release is shaped by a global community of contributors who chose to build something open rather than something profitable.

A2. The Licence — What It Actually Says

The Four Freedoms of Free Software

Richard Stallman's GNU Project defines four fundamental freedoms that free software must grant to every user. LibreOffice's licence — the Mozilla Public Licence version 2.0 (MPL 2.0) — must be evaluated against these four freedoms to determine whether it qualifies as genuine free software.

- **Freedom 0 — The freedom to run the program for any purpose.** LibreOffice can be installed and run by anyone, for any use: personal, educational, commercial, or governmental. There are no restrictions on purpose. This freedom is fully granted.
- **Freedom 1 — The freedom to study how the program works and adapt it.** The complete source code of LibreOffice is publicly available on GitHub and The Document Foundation's infrastructure. Any developer can read, study, fork, and modify it. This freedom is fully granted.
- **Freedom 2 — The freedom to redistribute copies.** LibreOffice can be redistributed freely, whether in original or modified form, provided the source of modifications is disclosed under MPL 2.0. This freedom is fully granted.
- **Freedom 3 — The freedom to distribute modified versions.** Developers may create and distribute modified versions of LibreOffice, including commercial distributions, as long as the modified source files remain under MPL 2.0. This freedom is fully granted.

LibreOffice grants all four freedoms. The software qualifies as free software in Stallman's precise sense: not free as in zero cost, but free as in user liberty.

Can a Company Take LibreOffice, Modify It, and Sell It Without Sharing Changes?

This is one of the most practically important questions about the MPL 2.0. The answer is nuanced. Under MPL 2.0, a company *can* integrate LibreOffice code into a proprietary product, provided it complies with the licence's file-level copyleft requirement. This means that any file originally licensed under MPL 2.0 that is modified *must* remain available under MPL 2.0. However, the company may combine those files with proprietary code in a larger executable without being required to release the proprietary code.

In practice, this is why MPL 2.0 is described as a 'weak copyleft' licence. It is more restrictive than MIT or Apache (which allow files to be relicensed entirely), but less restrictive than the GPL (which requires the entire combined work to be released under the GPL if distributed). For LibreOffice, this means commercial vendors — including collabora, which distributes Collabora

Online — can build commercial products on top of LibreOffice while keeping proprietary additions private, as long as any changes to LibreOffice's own files are disclosed.

GPL vs MIT: The Key Difference

The GNU General Public Licence (GPL) is a 'strong copyleft' licence. Any derivative work or software that incorporates GPL code and is distributed to others must itself be released under the GPL, with its full source code available. This 'viral' property ensures that the freedom granted to users cannot be stripped away by downstream distributors.

The MIT Licence is a 'permissive' licence. It imposes almost no restrictions: any developer can take MIT-licensed code, modify it, incorporate it into a proprietary product, and sell that product without releasing the source. The only obligation is to retain the original copyright notice.

For personal open-source projects where broad adoption matters most, MIT is often the pragmatic choice — it removes every barrier to use. For projects where community freedom and the prevention of proprietary capture are the primary values, the GPL is the better choice. The ideal choice depends on what the developer values: maximum adoption, or maximum freedom preservation.

'Free as in Free Beer' vs 'Free as in Freedom'

Richard Stallman's distinction — 'free as in speech, not free as in beer' — is one of the most important conceptual contributions to the open-source movement. 'Free beer' refers to zero cost: you receive something at no financial charge, but the provider still controls what it is, how it works, and whether it continues to exist. 'Free speech' refers to liberty: you have the right to examine, modify, share, and build upon what you receive.

LibreOffice illustrates both sides of this distinction. It is free of charge — anyone can download and install it without paying a single rupee or dollar. But more importantly, it is free as in freedom: the source code is public, the licence permits modification, and no company can suddenly make it proprietary or discontinue it, because the code belongs to the community. When Oracle tried to close off OpenOffice.org, the community was able to fork LibreOffice precisely because they had these freedoms. With a proprietary product, that option simply does not exist.

A3. The Ethics of Open Source

Should All Software Be Open Source?

The case for universal open-source software is compelling. When source code is available for public inspection, security vulnerabilities are more likely to be discovered and fixed quickly — the 'many eyes' principle. Software that embeds itself in critical infrastructure (hospital systems, election machinery, financial platforms) should arguably be inspectable by society. The concentration of power that comes with proprietary software — where a single vendor can lock millions of users into formats, pricing, and update schedules — is itself an ethical concern in a democratic society.

The case against is equally serious. Building and maintaining high-quality software is expensive. Proprietary licensing enables companies to generate the sustained revenue that funds large engineering teams, dedicated security research, and long-term product investment. A world where all software must be open source would make it considerably harder to build certain categories of software — not because open source is inherently inferior, but because the business models that sustain intensive software development become much harder to construct. Open source works spectacularly well when there is a large community of motivated contributors or when commercial entities have overlapping interests. It works less well when software requires deep, specialised investment with limited community overlap.

The more honest answer is that neither model is universally superior. The world benefits from a mixture: open-source foundations (operating systems, compilers, databases, web servers) that everyone builds on, combined with proprietary innovation layers that fund specialised development. What matters is that the underlying foundations — the soil in which all software grows — remain open and shared.

Is It Ethical to Build a Profitable Business on Free Community Labour?

Companies like Red Hat, Canonical, Google, and Meta have built extraordinarily profitable products and businesses on top of Linux and other FOSS projects contributed to largely by volunteers and independent developers. Red Hat was acquired by IBM for $34 billion — a figure that represents enormous value extracted from work that was given freely to the community.

The ethical question is real. Contributors to the Linux kernel, Apache, or LibreOffice often receive no financial compensation while corporations profit enormously from their work. However, the situation is more complex than straightforward exploitation. Many of these companies also contribute significantly back to the projects they use — Google employs full-time Linux kernel engineers, Red Hat employs more kernel contributors than any other single organisation, and Collabora employs LibreOffice developers. The licence structure explicitly permits commercial use, meaning contributors knowingly chose a model that allows corporate participation. The question of fairness is not resolved by these facts, but it is complicated by them.

What seems genuinely problematic is when a company takes disproportionately and contributes nothing back — not code, not financial support, not documentation, not community participation. The FOSS ecosystem depends on a rough norm of reciprocity. Companies that violate this norm may be operating within the law but are arguably behaving against the spirit of the community they depend on.

Open Source and the Question of Original Innovation

The phrase 'standing on the shoulders of giants' describes exactly what modern software development looks like. Every application built today runs on layers of open-source foundations: a Linux kernel, a compiler toolchain from GNU or LLVM, runtime libraries, databases, web servers. Developers do not rebuild these from scratch — they build on top of

them, which means they can move faster and create more sophisticated things than any single developer or team could produce alone.

Open source enables original innovation rather than undermining it. By removing the need to solve problems that have already been solved, it frees developers to focus on genuinely new problems. The developer who contributes to LibreOffice benefits from decades of word-processor, spreadsheet, and rendering research that others have already done. In turn, their contributions become part of the foundation for the next generation of developers. This is a fundamentally different model from proprietary development, where solved problems must be solved again and again by each competing company — an enormous collective waste of human engineering effort.

Part B — Linux Footprint (Unit 2)

This section documents how LibreOffice lives on a Linux system. All observations were made on Fedora 43 with LibreOffice version 26.2.0.3 installed via the default DNF package manager.

Installation Method

LibreOffice is available through Fedora's official package repositories and can be installed with a single DNF command:

```
sudo dnf install libreoffice
```

The package manager resolves all dependencies automatically, downloading and installing LibreOffice along with its required runtime libraries. Alternatively, LibreOffice can be downloaded as a pre-built binary tarball directly from libreoffice.org, or compiled from source — though the package manager route is standard for most users. On Debian/Ubuntu systems, the equivalent command uses APT: `sudo apt install libreoffice`.

Key Directories

Directory	Contents / Purpose
/usr/bin/libreoffice	Main executable launcher and component wrappers (swriter, scalc, simpress, etc.)
/usr/lib/libreoffice/	Core LibreOffice libraries, shared objects (.so files), and runtime components
/usr/share/libreoffice/	Shared data: templates, extensions, help files, and locale resources
/etc/libreoffice/	System-wide configuration files (if present; may be absent on minimal installs)
~/.config/libreoffice/	Per-user configuration, settings, extensions, and macro storage
/var/log/	System logs where DNF installation activity is recorded

User and Group — Security Implications

LibreOffice runs under the same user account that launches it — there is no separate system user or privileged service account. When a student opens LibreOffice Writer, the process runs with that student's own UID and GID, meaning it can only access files that the user can access. This is a deliberate security design: the application is sandboxed to the user's own permissions boundary. It cannot read /etc/shadow, write to system directories, or escalate privileges — which dramatically limits the damage a malicious document macro could cause.

This stands in contrast to services like Apache httpd, which run under a dedicated www-data user with minimal privileges, or MySQL, which runs as the mysql user. The principle in both cases is the same: *least privilege* — every process should run with the minimum permissions

required to perform its function.

Service Management

LibreOffice is not a background service in a standard desktop installation; it is an interactive application launched by the user on demand. However, LibreOffice can be run in headless server mode for document conversion and rendering tasks, which is common in enterprise and web application deployments:

```
# Start LibreOffice in headless mode for document conversion
libreoffice --headless --convert-to pdf document.odt

# Using LibreOffice as a listening socket server
libreoffice --headless --accept="socket,host=localhost,port=2002;urp;"

# Check if headless LibreOffice process is running
pgrep -a soffice
```

In server environments where LibreOffice runs continuously for automated document conversion, it can be managed as a systemd service with standard start/stop/restart/status commands.

Update Model

On Fedora, LibreOffice updates are delivered through the standard DNF update pipeline:

```
# Check for and install all available updates
sudo dnf update

# Update only LibreOffice
sudo dnf update libreoffice

# Check available versions
dnf info libreoffice
```

Security patches are typically released by The Document Foundation's security team, packaged by Fedora maintainers, and pushed through the repository infrastructure within days of disclosure. This transparent patch pipeline is one of the strongest arguments for open-source software in security-sensitive environments: vulnerabilities can be reported, reviewed, patched, and distributed to every user in the world without any single company needing to approve or delay the process.

Part C — The FOSS Ecosystem (Units 3 & 4)

Dependencies: What LibreOffice Is Built On

LibreOffice is one of the largest open-source desktop applications in existence, comprising several million lines of code. Its construction depends on a significant stack of open-source components:

- **Java Runtime Environment (OpenJDK):** Required for certain LibreOffice features including the HSQLDB database engine used in Base, and several macro and extension capabilities.
- **Python:** Used extensively for macro scripting and extension development. LibreOffice ships with a built-in Python interpreter for scripting automation.
- **Boost C++ Libraries:** Provides foundational data structures, algorithms, and utilities used across LibreOffice's codebase.
- **ICU (International Components for Unicode):** Powers LibreOffice's multilingual text handling, bidirectional text support, and locale-specific formatting across its 100+ language translations.
- **Cairo Graphics Library:** Used for high-quality 2D rendering of text, shapes, and drawings within documents.
- **HarfBuzz:** Open-source text shaping engine that correctly handles complex scripts like Devanagari, Arabic, and Thai.
- **Hunspell:** The open-source spell-checking library that powers spell checking across all LibreOffice applications, also used in Firefox and macOS.
- **libxml2 / libxslt:** XML parsing and XSLT transformation libraries that underpin LibreOffice's ODF (Open Document Format) file handling.

What LibreOffice Has Enabled and Inspired

LibreOffice's influence on the open-source ecosystem extends well beyond its own user base:

- **Collabora Online:** A cloud-based document editor built on LibreOffice technology that powers self-hosted office collaboration for organisations that do not wish to use Google Workspace or Microsoft 365. It integrates with Nextcloud, OwnCloud, and enterprise Kubernetes deployments.
- **ONLYOFFICE:** Though independently developed, ONLYOFFICE's rise as an open-source alternative to Google Docs was directly enabled by the market space that LibreOffice opened and the ODF standard it helped establish.
- **OpenDocument Format (ODF):** LibreOffice is the primary reference implementation of ODF, the ISO-standardised open document format. Its advocacy helped push the Indian government, several EU member states, and dozens of municipal governments to adopt ODF as their mandated format for public documents — breaking vendor lock-in at the

governmental level.

- **LibreOffice Online / COOL:** The Collabora Online Development Edition (CODE), a community version of Collabora Online, is maintained as part of the LibreOffice project and shows how the desktop application has evolved toward a cloud-native future.

LibreOffice and the Web — LAMP Context

LibreOffice does not sit directly in the LAMP (Linux, Apache, MySQL, PHP) stack, but it occupies a critical adjacent role in server-side document processing workflows. Many web applications need to generate, convert, or render documents — invoices, reports, certificates, exported spreadsheets. These workflows commonly combine LAMP components with LibreOffice:

- A PHP or Python web application (running on Apache) generates an ODT or DOCX file.
- A background process calls LibreOffice in headless mode to convert that file to PDF.
- The PDF is stored in a MySQL/MariaDB-tracked file store and served to the end user.

LibreOffice's headless conversion capability makes it an invisible but essential component in many document-heavy web application backends — particularly in sectors like legal, finance, and education where PDF generation from dynamic templates is a core requirement.

Community and Governance

LibreOffice is governed by The Document Foundation (TDF), a German charitable foundation (Stiftung) with legal status. The foundation's structure is designed to prevent any single company from dominating the project:

- **Board of Directors:** An elected board of seven trustees governs TDF's strategic direction.
- **Engineering Steering Committee:** Oversees technical decisions and release engineering.
- **Community participation:** Anyone can contribute code, documentation, translations, bug reports, or design work. The contribution process is documented and open.
- **Mailing lists and IRC/Matrix:** Active developer mailing lists (libreoffice-dev) and Matrix channels handle day-to-day development discussion.
- **LibreOffice Conference:** An annual conference held in different European cities each year, bringing together developers, users, and advocates from around the world.
- **Bug tracker:** bugs.documentfoundation.org hosts the public Bugzilla instance where anyone can file or comment on bugs.

Part D — Open Source vs Proprietary (Critical Analysis)

The comparison below is made between LibreOffice (open source, MPL 2.0) and Microsoft Office 365 (proprietary, subscription-based), the dominant commercial alternative.

Dimension	LibreOffice	Microsoft Office 365
Cost	Free to download, use, and redistribute. No subscrip	[illegible]
Security — Code Auditabili	yFull source code publicly available. Security research	[illegible]
Feature Completeness	Highly capable for standard office tasks. Occasional	[illegible]
Support and Reliability	Community support via forums, mailing lists, and doc	[illegible]
Freedom to Modify	Full freedom to modify, fork, and redistribute. Organi	[illegible]
Community vs Corporate C	nGtroovlerned by an independent non-profit foundation (T	[illegible]
Overall Verdict	Recommended for: educational institutions, governm	[illegible]

Conclusion: Real-World Deployment Verdict

Based on this audit, LibreOffice is confidently recommended for real-world deployment in educational institutions, government departments, small-to-medium enterprises, and any organisation where financial sustainability and digital sovereignty are priorities. The total cost saving over five years for even a 100-person organisation — compared to Microsoft 365 licensing — runs into millions of rupees. The software is feature-complete for the vast majority of daily office tasks. The ODF standard it implements ensures long-term document accessibility without vendor lock-in.

The honest caveat is this: organisations with complex Microsoft Office macro workflows, deep Azure ecosystem integration, or a workforce trained exclusively on Microsoft interfaces will face real transition costs. LibreOffice is not always a drop-in replacement for advanced Excel models or complex Word templates with embedded VBA. For those environments, a hybrid deployment — LibreOffice for general use, Microsoft Office for power users — may be the most pragmatic path.

As for personal contribution: having studied LibreOffice's codebase structure, its contribution guidelines, and the active bug tracker, contributing to LibreOffice is entirely accessible for a motivated student. The project maintains a list of 'Easy Hacks' — tagged bugs and feature requests specifically chosen as good entry points for new contributors. The community is welcoming, the documentation is thorough, and the opportunity to contribute to software used by hundreds of millions of people is an extraordinary one.

Shell Script Tasks (40 Marks — 8 Marks Each)

Script 1 — System Identity Report

This script introduces the Linux system like a welcome screen. It collects key system information using command substitution and displays it in a formatted banner. The script identifies the student, the chosen open-source software, and provides a brief note about Fedora's licensing.

Concepts Used

- Variables — storing student name, registration number, and software choice
- Command substitution `$( )` — capturing output of uname, whoami, uptime, date
- echo — formatted output with separators
- Reading system files — /etc/fedora-release for the distribution name

Script Code

```
#!/bin/bash
# Script 1: System Identity Report
# Author: Hiten Gogia
# Course: Open Source Software

# --- Variables ---
STUDENT_NAME="Hiten Gogia"
REG_NO="24BCE10832"
SOFTWARE_CHOICE="LibreOffice"

# --- System Information ---
DISTRO=$(cat /etc/fedora-release)
KERNEL=$(uname -r)
USER_NAME=$(whoami)
HOME_DIR=$HOME
UPTIME=$(uptime -p)
CURRENT_DATE=$(date)

# --- Display ---
echo "========================================="
echo " Open Source Audit — $STUDENT_NAME"
echo "========================================="
echo "Chosen Software : $SOFTWARE_CHOICE"
echo "Distribution    : $DISTRO"
echo "Kernel Version  : $KERNEL"
echo "Current User    : $USER_NAME"
echo "Home Directory  : $HOME_DIR"
echo "System Uptime   : $UPTIME"
echo "Date and Time   : $CURRENT_DATE"
echo ""
echo "License: Fedora Linux is distributed under"
echo "various free software licenses, mainly the GPL."
echo "========================================
```

Expected Output

```
========================================
```

```
 Open Source Audit — Hiten Gogia
========================================
Chosen Software : LibreOffice
Distribution    : Fedora release 43 (Rawhide)
Kernel Version  : 6.x.x-300.fc43.x86_64
Current User    : hiten
Home Directory  : /home/hiten
System Uptime   : up 2 hours, 14 minutes
Date and Time   : Mon Mar 25 10:42:31 IST 2026

License: Fedora Linux is distributed under
various free software licenses, mainly the GPL.
========================================
```

Script 2 — FOSS Package Inspector

This script checks whether LibreOffice is installed on the system using the RPM package manager. If the package is found, it retrieves and displays detailed package metadata. A case statement then prints a short philosophy note tied to the package name, contextualising the open-source significance of the software.

Concepts Used

- if-then-else — conditional check using rpm -q to test for package presence
- rpm -qi with grep — querying and filtering package metadata
- case statement — pattern-matching on the package name to print philosophy notes
- Pipe — connecting rpm output to grep for targeted field extraction

Script Code

```
#!/bin/bash
# Script 2: FOSS Package Inspector
# Author: Hiten Gogia

PACKAGE="libreoffice"

echo "==============================================="
echo "        FOSS PACKAGE INSPECTOR"
echo "==============================================="

if rpm -q $PACKAGE &>/dev/null; then
    echo "$PACKAGE is installed."
    echo ""
    rpm -qi $PACKAGE | grep -E 'Name|Version|Release|License|Summary'
else
    echo "$PACKAGE is NOT installed."
fi

echo ""
echo "PHILOSOPHY NOTE"
echo "-------------- "

case $PACKAGE in
    libreoffice)
        echo "LibreOffice: a community-driven office suite that protects user freedom."
        ;;
    firefox)
```

```
        echo "Firefox: a browser created to keep the web open and independent."
        ;;
    git)
        echo "Git: version control built to support open collaboration."
        ;;
    vlc)
        echo "VLC: media software that proves open source can be universal."
        ;;
    *)
        echo "Unknown package."
        ;;
esac
```

Script 3 — Disk and Permission Auditor

This script loops through a predefined list of critical system directories and reports the size and ownership/permission string for each. It then checks for the existence of the LibreOffice system configuration directory. This script demonstrates array iteration and command output parsing using awk and cut.

Concepts Used

- Arrays — storing directory paths in a Bash array
- for loop — iterating over each directory in the array
- if statement — checking directory existence with -d
- du — measuring directory disk usage
- ls -ld with awk — extracting permission string, owner, and group
- cut — trimming du output to just the size field

Script Code

```
#!/bin/bash
# Script 3: Disk and Permission Auditor
# Author: Hiten Gogia

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "=============================================="
echo "      DISK AND PERMISSION AUDITOR"
echo "=============================================="

for DIR in "${DIRS[@]}"
do
    if [ -d "$DIR" ]; then
        PERMS=$(ls -ld "$DIR" | awk '{print $1, $3, $4}')
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
        echo "$DIR"
        echo "Permissions : $PERMS"
        echo "Size        : $SIZE"
        echo "------------------------------------ "
    else
        echo "$DIR does not exist."
    fi
done
```

```
echo ""
echo "LibreOffice Configuration Directory"
echo "---------------------------------- "

CONFIG="/etc/libreoffice"

if [ -d "$CONFIG" ]; then
    ls -ld "$CONFIG"
else
    echo "$CONFIG not found on this system."
fi
```

Script 4 — Log File Analyzer

This script accepts a log file path as a command-line argument and counts how many lines contain a specified keyword (defaulting to 'error'). It then prints the count and the last five matching lines. This demonstrates practical log analysis — a common sysadmin task — using grep, wc, and tail.

Concepts Used

- Command-line arguments — \$1 for the log file path, \$2 for the keyword
- Default variable values — \${2:-"error"} sets a fallback if no keyword is given
- File existence check — [! -f] with exit on failure
- grep with -i flag — case-insensitive keyword search
- wc -l — counting matching lines
- tail -5 — displaying the five most recent matches

Script Code

```
#!/bin/bash
# Script 4: Log File Analyzer
# Author: Hiten Gogia

LOGFILE=$1
KEYWORD=${2:-"error"}

echo "==============================================="
echo "               LOG FILE ANALYZER"
echo "==============================================="

if [ ! -f "$LOGFILE" ]; then
    echo "Error: File not found."
    exit 1
fi

COUNT=$(grep -i "$KEYWORD" "$LOGFILE" | wc -l)

echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"
echo ""
echo "Last 5 matching lines:"
grep -i "$KEYWORD" "$LOGFILE" | tail -5
```

Usage Examples

```
# Analyse system messages log for errors (requires sudo)
sudo ./script4_log_analyzer.sh /var/log/messages

# Search for 'warning' instead of the default 'error'
sudo ./script4_log_analyzer.sh /var/log/messages warning

# Analyse a custom log file
./script4_log_analyzer.sh /path/to/app.log critical
```

Script 5 — Open Source Manifesto Generator

This is the most creative script in the project. It prompts the user with three questions about their relationship with open-source software, captures their answers interactively, and assembles a personalised open-source philosophy statement. The manifesto is written to a text file named after the current user and then displayed on screen.

Concepts Used

- read with -p flag — interactive user input with inline prompts
- Variables — storing user responses and assembled text
- String concatenation — building a paragraph from variable values
- File redirection — > for file creation, >> for appending lines
- date command — embedding the generation timestamp in the output
- cat — displaying the completed manifesto on screen after saving

Script Code

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator
# Author: Hiten Gogia
echo "Answer three questions to generate your manifesto."
echo ""

read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does freedom mean to you? " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD

DATE=$(date '+%d %B %Y')
OUTPUT="manifesto_$(whoami).txt"

echo "Open Source Manifesto" > $OUTPUT
echo "Generated on: $DATE" >> $OUTPUT
echo "" >> $OUTPUT
echo "I use $TOOL every day because open-source software gives me $FREEDOM." >> $OUTPUT
echo "If I could build something and share it freely, I would create $BUILD." >> $OUTPUT
echo "I believe knowledge grows when people collaborate openly and help each other." >> $OUTPUT

echo ""
echo "Manifesto saved to $OUTPUT"
echo ""
cat $OUTPUT
```

Sample Output

```
Answer three questions to generate your manifesto.

1. Name one open-source tool you use every day: LibreOffice
2. In one word, what does freedom mean to you? independence
3. Name one thing you would build and share freely: a free education platform

Manifesto saved to manifesto_hiten.txt

Open Source Manifesto
Generated on: 25 March 2026

I use LibreOffice every day because open-source software gives me independence.
If I could build something and share it freely, I would create a free education platform.
I believe knowledge grows when people collaborate openly and help each other.
```

Conclusion

This project has been an exercise in looking beneath the surface of software that most users take for granted. LibreOffice is not simply a free alternative to Microsoft Office — it is a statement about how knowledge and tools should be distributed in a just society. Its origin story, rooted in a community's refusal to accept corporate capture of their work, is one of the clearest demonstrations in computing history of what the open-source movement actually stands for.

The licence analysis revealed that MPL 2.0 is a thoughtfully designed instrument — permissive enough to allow commercial participation without being so permissive that corporate actors can strip freedoms from future users. The ethical reflection surfaced genuinely difficult questions: about the sustainability of volunteer labour, about the responsibilities of corporations that profit from community work, and about the right balance between open foundations and proprietary innovation layers.

The Linux footprint section demonstrated that LibreOffice behaves exactly as a well-designed Unix application should: installing into standard directories, running under user permissions, updating through transparent package management, and offering headless operation for server-side integration.

The ecosystem mapping revealed that LibreOffice is not an island — it sits at the centre of a network of dependencies, spin-offs, standards, and community institutions that collectively represent decades of shared human effort. Its influence on the ODF standard alone has shaped government document policy in dozens of countries.

The five shell scripts, modest as they are, demonstrate something important: that the tools of open-source computing — bash, grep, rpm, awk — are themselves open, inspectable, and composable. The philosophy of LibreOffice and the tools used to audit it share the same foundation: that transparency, sharing, and community are not just nice-to-haves. They are what makes the whole system work.